

Pebble /set-renewal-info/ — exact management API for a hermetic ARI-forced renewal

Revision: 1 **Last modified:** 2026-07-01T11:15:00Z **Authority:** Inherits constitution/Constitution.md per §11.4.35. §11.4.150 deep-research for LE Phase-5 (renewal/rotation). Source-verified from Pebble tags v2.8.0 + v2.10.1 (fetched 2026-07-01).

1. The request (verbatim from source; identical v2.8.0 → v2.10.1)

- **Method + path:** POST /set-renewal-info/
- **Listener:** the **management** listener (managementListenAddress: 0.0.0.0:15000), HTTPS signed by Pebble's minica → curl -k. Registered on the ManagementHandler() mux (same as /roots/0, /intermediates/0), NOT the ACME mux; no auth → keep :15000 loopback-bound.
- **Handler body** decodes exactly two fields (Go struct has NO json tags → case-insensitive; use canonical names):

```
{
  "Certificate": "-----BEGIN CERTIFICATE-----\n...leaf...\n-----END\nCERTIFICATE-----\n",
  "ARIResponse": "{\"suggestedWindow\":{\"start\":\"<past>\",
    \"end\":\"<past>\"}}"
```

- Certificate = the **leaf in PEM**; must decode to a CERTIFICATE block or → 400.
- ARIResponse = a **string stored + served back verbatim** — Pebble never parses it. Put the RFC 9773 renewalInfo JSON here.
- **Responses:** 200 (empty body) · 400 MalformedProblem (bad JSON/PEM) · **404** when the serial is unknown to this Pebble boot (Pebble regenerates CA + serial ledger per boot — send the leaf issued in the CURRENT boot).

2. Cert identification — the whole leaf PEM, keyed by serial (NOT an ARI CertID)

Pebble does `pem.Decode` → `x509.ParseCertificate` → `db.SetARIResponse(cert.SerialNumber, ARIResponse)`. You send the leaf PEM; Pebble derives the serial. The RFC 9773 CertID (`base64url(AKI).base64url(serial)`) is only the CLIENT GET path component, not the management POST.

3. Values that move the window to NOW

The GET handler short-circuits to the override when set, with a fixed `Retry-After: 21600` (6h). Set `ARIResponse` to a suggested window fully in the past (mirrors Pebble's `RenewalInfoImmediate`):

```
{"suggestedWindow":{"start":"<now-2h RFC3339Z>","end":"<now-1h RFC3339Z>"}}
```

`certmagic v0.21.3` `certNeedsRenewal: cutoff := ari.SelectedTime - RenewCheckInterval; if now.After(cutoff) return true` — a past window ⇒ renew on the next ARI *evaluation*.

4. Flag / env compat 2.10.1 vs 2.6.0

- **CLI flags unchanged** (`-config`, `-dnsserver`, `-strict`, `-version`) — BUT in 2.8+ `-strict` is a **boolean** flag: the old `-strict false` is invalid (invalid command line arguments: `false`). Drop it (strict defaults `false`). (*Conductor live-confirmed on 2.10.1.*)
- **Env unchanged:** `PEBBLE_VA_ALWAYS_VALID` (0 = real validation), `PEBBLE_VA_NOSLEEP`, `PEBBLE_WFE_NONCEREJECT` all read with same semantics.
- **Config-file schema changed:** 2.6.0 top-level `certificateValidityPeriod` → 2.10.1 profiles map (`default.validityPeriod 7776000 = 90d`; `shortlived.validityPeriod`). The old key is silently ignored (unknown keys tolerated). Recommendation: for the rotation container use the image's bundled `/test/config/pebble-config.json` (don't override validity).

5. Exact curl (management :15000, -k)

```
openssl s_client -connect 127.0.0.1:<caddy-https> -servername  
    proxy.hermetic.test </dev/null 2>/dev/null | openssl x509  
    > leaf.pem  
serial_before=$(openssl x509 -in leaf.pem -noout -serial | cut  
    -d= -f2)
```

```

start=$(date -u -d '-2 hours' +%Y-%m-%dT%H:%M:%SZ); end=$(date -u
-d '-1 hour' +%Y-%m-%dT%H:%M:%SZ)
ari=$(printf '{"suggestedWindow":{"start":"%s","end":"%s"}}'
"$start" "$end")
jq -n --rawfile cert leaf.pem --arg ari "$ari" '{Certificate:
$cert, ARIResponse:$ari}' \
| curl -k -sS -X POST -H 'Content-Type: application/json' --data-
binary @- \
https://127.0.0.1:15000/set-renewal-info/ -w '%{http_code}
\n' # expect 200

```

6. Conductor live-verification 2026-07-01 (§11.4.6) — the residual blocker is certmagic, not Pebble

The conductor booted Pebble 2.10.1 + Caddy 2.8.4 and ran this end-to-end:

- Pebble 2.10.1 boots cleanly once `-strict false` is dropped; `/set-renewal-info/` exists.
- The POST returns **200**, and Pebble's ARI GET at `/draft-ietf-acme-ari-03/renewalInfo/<CertID>` returns **exactly the posted past window** (confirmed for the served leaf's computed CertID). **So the Pebble override is FACT-verified working.**
- BUT **Caddy did NOT renew.** certmagic v0.21.3 caches the ARI response persistently (in the cert's `issuer_data` under the `/data` volume, `Retry-After 6h`). **Neither POST /load NOR a Caddy restart forces a network re-fetch** — both read the cached (default, future) window. The override never reaches certmagic within the test window.

Conclusion (honest boundary §11.4.6): the deterministic renewal trigger is proven at the Pebble layer but blocked at the certmagic-v0.21.3 (Caddy 2.8.4) layer by a persistent 6h ARI cache with no force-refresh. A fast, zero-downtime, deterministic hermetic renewal requires **Caddy ≥ 2.11.0 / certmagic ≥ v0.25.1** (which also fixes the #354 panic) — and that bump requires adapting the custom `challtestsrv` DNS provider from `libdns v0.2.x` to **libdns v1.0.0** (typed-RR API; Caddy ≥2.10). That module adaptation + image rebuild is the Phase-5 unblocking sub-project. Wiping `/data` forces a re-ISSUE (new serial) but with downtime — proves rotation, not zero-downtime renewal.

Sources verified 2026-07-01

- `pebble@v2.8.0/wfe/wfe.go`, `pebble@v2.10.1/wfe/wfe.go` · `/core/types.go` · `/cmd/pebble/main.go` · `/va/va.go` · `/ca/ca.go` · `/test/config/pebble-config.json` (raw.githubusercontent.com, pinned tags)

- Pebble PR #501 (adds /set-renewal-info/, fixes #486, → v2.8.0): <https://github.com/letsencrypt/pebble/pull/501>
- docs/research/letsencrypt_renewal_20260701/ANALYSIS.md Rev 3 (certmagic v0.21.3 certNeedsRenewal; conductor live FACTs)
- RFC 9773 (suggestedWindow): <https://datatracker.ietf.org/doc/rfc9773/>